TRIBHUVAN UNIVERSITY
INSTITUTE OF ENGINEERING

Khwopa College Of Engineering
Libali, Bhaktapur
Department of Computer Engineering

A PROPOSAL ON

SPAM MESSAGE DETECTION USING MACHINE LEARNING

Submitted in partial fulfillment of the requirements for the degree

BACHELOR OF COMPUTER ENGINEERING

Submitted by

Aayush Dahal	KCE/076/BCT/002
Jayan Ghimire	KCE/076/BCT/018
Samik Bhattarai	KCE/076/BCT/036
Sankalpa Kandel	KCE/076/BCT/040

Khwopa College Of Engineering
Libali, Bhaktapur
2022-23

Acknowledgement

We would like to express our deepest gratitude to our department head **Er.Dinesh Man Gothe** for contributing his valuable time and efforts in helping us out with this project. His suggestions and feedback have helped us a lot in improving the quality of the project. We would also like to thank **Er.Bindu Bhandari** for her constant encouragement and support throughout the project. Lastly, We like to thank all our supporters and friends who have motivated us to fulfill our project within the timeline.

Aayush Dahal	KCE/076/BCT/002
Jayan Ghimire	KCE/076/BCT/018
Samik Bhattarai	KCE/076/BCT/036
Sankalpa Kandel	KCE/076/BCT/040

Abstract

The spam detection is a big issue in mobile message communication due to which mobile message communication is insecure. In order to tackle this problem, an accurate and precise method is needed to detect the spam in mobile message communication. We proposed the applications of the machine learning-based spam detection method for accurate detection. In this technique, machine learning classifiers such as Logistic regression (LR), K-nearest neighbor (K-NN), decision tree (DT) and other algorithms are used for classification of ham and spam messages in communication devices. The data set is used for testing the method. The dataset is split into two categories for training and testing the research. Additionally, the proposed method performance will be good as compared with the existing state-of-the-art methods.

Keywords: *Spam, Communication, Precise, Logistic Regression, K-Nearest Neighbor, Ham, Spam, Dataset*

Contents

List of Tables

List of Figures

List of Symbols and Abbreviation

API	Application Programming Interface
BVN	Bank Verification Number
EDA	Exploratory Data Analysis
ETC	Extra Tress Classifier
GBDT	Gradient Boosting Decision Trees
KNN	K-Nearest Neighbours
LCS	Longest Common Subsequence
LD	Levenshtein Distance
LR	Logistic Regresssion
ML	Machine Learning
NB	Naïve Bayes
NTLK	Natural Language Toolkit
RF	Random Forest
SMTP	Simple Mail Tranfer Protocol
SVM	Support Vector Machine
UI	User Interface
URL	Uniform Resource Locator

Chapter 1

Introduction

1.1 Background Introduction

In recent times, unwanted commercial bulk emails called spam has become a huge problem on the internet. The person sending the spam messages is referred to as the spammer. Such a person gathers email addresses from different websites, chatrooms, and viruses [1]. Spam prevents the user from making full and good use of time, storage capacity and network bandwidth. The huge volume of spam mails flowing through the computer networks have destructive effects on the memory space of email servers, communication bandwidth, CPU power and user time. According to report from Kaspersky lab, in 2015, the volume of spam emails being sent reduced to a 12-year low. Spam email volume fell below 50% for the first time since 2003. In June 2015, the volume of spam emails went down to 49.7% and in July 2015 the figures was further reduced to 46.4% according to anti-virus software developer Symantec. This decline was attributed to reduction in the number of major botnets responsible for sending spam emails in billions. Malicious spam email volume was reported to be constant in 2015. The figure of spam mails detected by Kaspersky Lab in 2015 was between 3 million and 6 million. Conversely, as the year was about to end, spam email volume escalated. Further report from Kaspersky Lab indicated that spam email messages having pernicious attachments such as malware, ransomware, malicious macros, and JavaScript started to increase in December 2015. That drift was sustained in 2016 and by March of that year spam email volume had quadrupled with respect to that witnessed in 2015. In March 2016, the volume of spam emails discovered by Kaspersky Lab is 22,890,956. By that time the volume of spam emails had skyrocketed to an average of 56.92% for the first quarter of 2016. Latest statistics shows that spam messages accounted for 56.87% of e-mail traffic worldwide and the most familiar types of spam emails were healthcare and dating spam. Spam results into unproductive use of resources on Simple Mail Transfer Protocol (SMTP) servers since they have to process a substantial volume of unsolicited emails.

1.2 Problem Statement

The menace of spam messages is on the increase on yearly basis and is responsible for over 77% of the whole global message traffic. Users who receive spam emails that they did not request, find it very irritating. It is also resulted to untold financial loss to many users who have fallen victim of internet scams and other fraudulent practices of spammers who send messages and emails pretending to be from reputable companies with the intention to persuade individuals to disclose sensitive personal information like Passwords, Bank Verification Number (BVN) and Credit Card numbers. To effectively handle the threat posed by email spams, leading email providers such as Gmail, Yahoo mail and Outlook have employed the combination of different machine learning (ML) techniques such as Neural Networks in its spam filters. These ML techniques have the capacity to learn and identify spam mails and phishing messages by analyzing loads of such messages throughout a vast collection of computers. Since machine learning have the capacity to adapt to varying conditions, Gmail and Yahoo mail spam filters do more than just checking junk emails using pre-existing rules. They generate new rules themselves based on what they have learnt as they continue in their spam filtering operation.

1.3 Obejctive

The main aim of this project is:

- To test various models for spam message detection.
- To reduce effort and time to detect spam messages.
- To classify messages as spam or ham as it is a major problem.

Chapter 2

Literature Review

In the paper [2], authors have highlighted several features contained in the email header which will be used to identify and classify spam messages efficiently. Those features are selected based on their performance in detecting spam messages. This paper also communalizes each feature contains in Yahoo mail, Gmail and Hotmail so a generic spam messages detection mechanism could be proposed for all major email providers.

In the paper [3], author collected email dataset from the online available websites and used Naïve Bayes for filtering of emails. He proposed a hybrid approach using secure hash method and Naive Bayes to filter email data but could not provide information regarding the misuse of storage resources and network bandwidth. By using Secure Hash Algorithm, the email is considered as a message M due to a generated function. The message M is further classified into S and L where L stands for ham email or genuine email and on the other hand S stands for spam email.

In the paper [4],authors described about cyber-attacks .Phishers and malicious attackers are frequently using email services to send false kinds of messages by which target user can lose their money and social reputations. These results into gaining personal credentials such as credit card number, passwords and some confidential data. In This paper, authors have used Bayesian Classifiers considering every single word in the mail constantly adapting to new forms of spam.

In the paper [5], authors worked on the solution for combining classification technique to get better result for spam filtering. The author took help of data mining and by using that, collected all the information regarding success, current problems and previous failures of spam filtering. The method was based on binary classification where 1 was used for Spam email and 0 was used for Ham emails. They combined the 2 method that is Machine Learning and Knowledge Engineering for the filtering of emails. The performance of the proposed method was very poor on the combined KNN and SVM algorithm.

In the paper [6], authors investigated the use of string matching algorithms for spam email detection. Particularly this work examines and compares the efficiency of six well-known string-matching algorithms, namely Longest Common Subsequence (LCS), Levenshtein Distance (LD), Jaro , Jaro - Winkler, Bi-gram, and TFIDF on two various datasets which are Enron corpus and CSDMC2010

spam dataset. They observed that Bi-gram algorithm performs best in spam detection in both datasets.

Chapter 3

Requirement Analysis

3.1 Software Requirement

Software requirement for our proposed system includes:

1. Python
2. Google Colaboratory
3. Git Hub
4. CSS
5. Star UML
6. Jupyter Notebook
7. Overleaf
8. Microsoft Teams
9. Google Drive
10. Javasript
11. HTML
12. Click Up

3.2 Hardware Requirement

Hardware requirement for our system are:

3.2.1 Graphical Processing Unit

For training our model, we will be using GPU provided by google colab.

3.2.2 Computer

Working PC with minimum Intel-i5 processor and internet connection

3.3 Functional Requiremtn

Followings can be listed as the functional requirements of our Spam Detection System:

- The system should be able to determine whether a message is spam or not.
- The system should be adaptable and elaborated from the point of view of the user
- The system should accept both url and text message as a form of input
- The system should be able to filter raw input by tokenizing,stemming,removing special characters and stop words

3.4 Non-Functional Requirement

The following non-functional requirements should be met by our project on Spam Detection System:

3.4.1 Reliability

As the system is highly reliable,the system will be able to run without fail in a specific environment.

3.4.2 Easy to use

The system is easy to use as user is just required to enter the message or ea valid url and click before displaying the result. new input data and is scalable to millions of data points.

3.4.3 Performance

Performance of the system depends on how fast the system gives the result. After its completion, the system will be able to detect spam messages quickly.

3.4.4 Portability

Spam message detector is portable and it is easy to integrate into any web application or mobile application by the use of the REST API's made.

3.4.5 Accuracy

The system should be able to tell whether the message is spam or ham. The system willbe capable of accurately detecting spam messages.

3.5 Feasibility Study

The following points describes the feasibility of the project.

3.5.1 Economic Feasibility

We want to employ the majority of open-source tools,libraries,and frameworks available on the internet. As a result, there is no investement of any budget in this project.

3.5.2 Technical Feasibility

The technology required for developing the project is easily available. Softwares like Python,Javascript,Jupyter Notebook are easily available which makes the develpoment, updating and upgrading the system very easy.

3.5.3 Operational Feasibility

The system working is quite easy to use and learn due to its simple but attractive interface. User requires no special training for operating the system.

3.5.4 Legal Feasibility

In this project,we are using free and open source dataset available on the internet.Also we will be developing this project without violating any laws and rules of the country so this project is legally feasible

3.5.5 Space Feasibility

We are attempting to keep this project as minimal as possible in order to save disk space. To reduce space complexity ,it will be preferable to use fewer libraries and framework.

Chapter 4

System Design and Architecture

We developed a fack message detection system which takes text and url as its input and process it to detect message as spam or ham.

4.1 Use Case Diagram

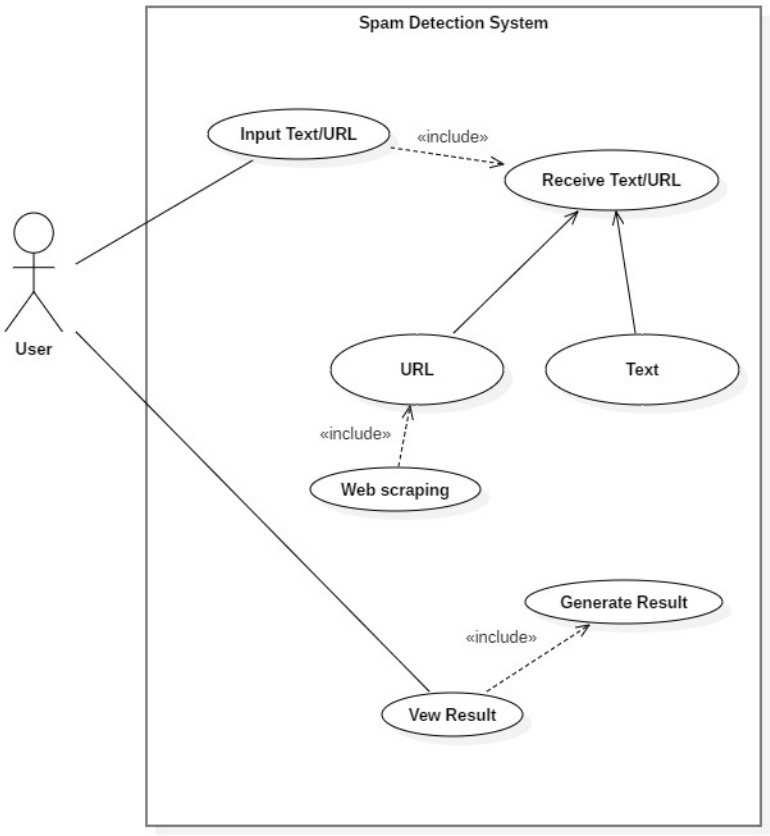

Figure 4.1: System Use Case Diagram

4.2 System Block

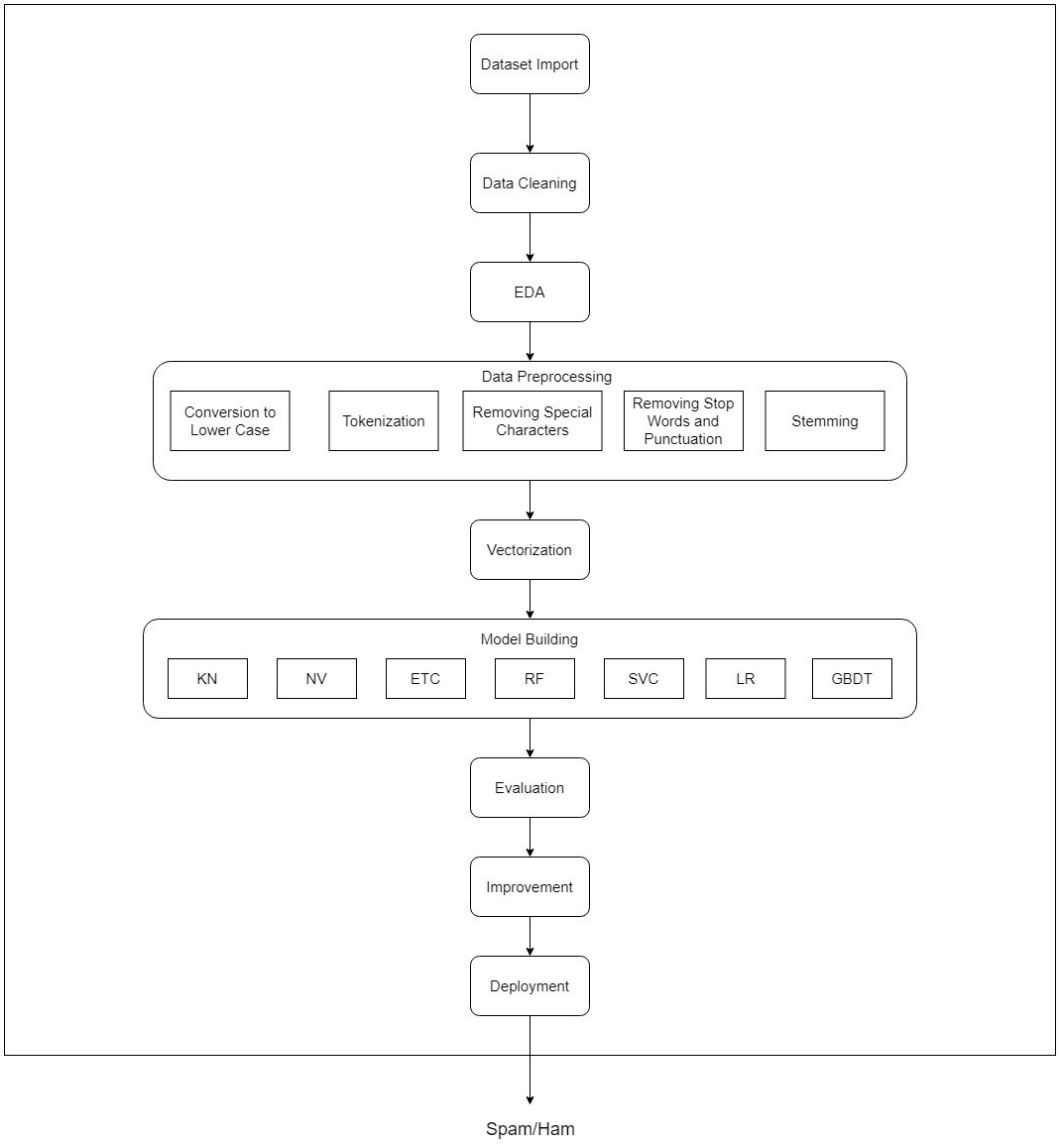

Figure 4.2: System Block Diagram

Chapter 5

Methodology

5.1 Software Development Approach

In this project, we will be using Iterative Waterfall Methodology as our software development model. Implementing our system in website was quite a challenging task which can be made easy by dividing project with a small section and developing it to a fully functioning project. When developing this project, different tasks will be assigned to team members and progress will be recorded accordingly. Our proposed system development approach is shown in figure 5.1

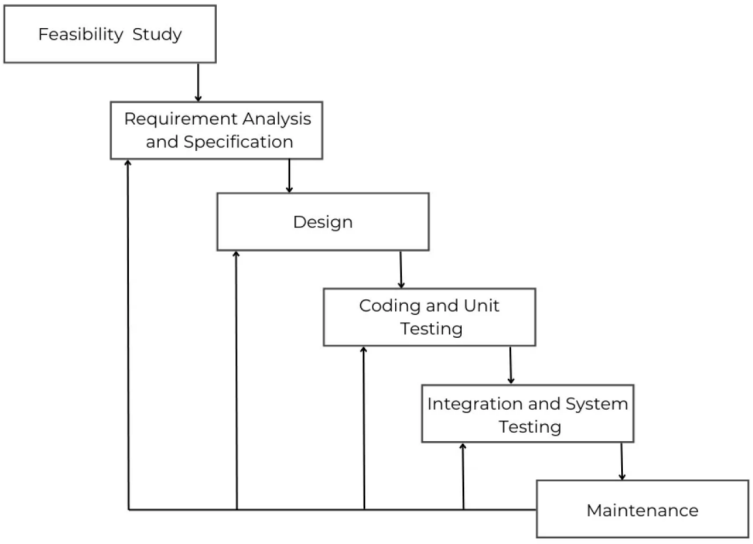

Figure 5.1: Iterative Waterfall Model for Software Development

5.2 Data Collection

We will be collecting data from open source plarform Kaggle which provided us with a data set of 5572 messages which were both spam and ham messages. This dataset will be used to train our model and also test the accuracy.Some portion of dataset is shown in figure 5.2

	v1	v2	Unnamed: 2	Unnamed: 3	Unnamed: 4
0	ham	Go until jurong point, crazy.. Available only ...	NaN	NaN	NaN
1	ham	Ok lar... Joking wif u oni...	NaN	NaN	NaN
2	spam	Free entry in 2 a wkly comp to win FA Cup fina...	NaN	NaN	NaN
3	ham	U dun say so early hor... U c already then say...	NaN	NaN	NaN
4	ham	Nah I don't think he goes to usf, he lives aro...	NaN	NaN	NaN

Figure 5.2: Sample Data Set

5.3 Data Cleaning

We can see in Figure 5.3 that in the column Unnamed: 2,3 and 4 most of the value are null values which are useless so we drop those columns and the final result is shown in Figure 5.4 after dropping the columns.

```
Data columns (total 5 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   v1          5572 non-null   object
 1   v2          5572 non-null   object
 2   Unnamed: 2  50 non-null     object
 3   Unnamed: 3  12 non-null     object
 4   Unnamed: 4  6 non-null      object
dtypes: object(5)
```

Figure 5.3: Before Dropping the columns

	v1	v2
1947	ham	The battery is for mr adewale my uncle. Aka Egbon
2712	ham	Hey you still want to go for yogasana? Coz if ...
4428	ham	Hey they r not watching movie tonight so i'll ...
3944	ham	I will be gentle princess! We will make sweet ...
49	ham	U don't know how stubborn I am. I didn't even ...

Figure 5.4: After Dropping the columns

5.4 EDA

Exploratory data analysis (EDA) is used to analyze and investigate data sets and summarize their main characteristics, often employing data visualization methods.

It helps determine how best to manipulate data sources to get the answers you need, making it easier to discover patterns, spot anomalies, test a hypothesis, or check assumptions.The main purpose of EDA is to help look at data before making any assumptions.

5.4.1 Tokenization

The unlabeled and unprepared data is fed into the tokenization engine. There are two types of tokenization; one sentence level tokenization and another word level tokenization. Following example shows both the sentence level tokenization as well as world level tokenization.

msg_lower	msg_tokenied
go until jurong point crazy available only in bugis n great world la e buffet cine there got amore wat	[go, until, jurong, point, crazy, available, only, in, bugis, n, great, world, la, e, buffet, cine, there, got, amore, wat]
ok lar joking wif u oni	[ok, lar, joking, wif, u, oni]
free entry in 2 a wkly comp to win fa cup final tkts 21st may 2005 text fa to 87121 to receive entry questionstd txt ratetcs apply 08452810075over18s	[free, entry, in, 2, a, wkly, comp, to, win, fa, cup, final, tkts, 21st, may, 2005, text, fa, to, 87121, to, receive, entry, questionstd, txt, ratetcs, apply, 08452810075over18s]
u dun say so early hor u c already then say	[u, dun, say, so, early, hor, u, c, already, then, say]
nah i dont think he goes to usf he lives around here though	[nah, i, dont, think, he, goes, to, usf, he, lives, around, here, though]

Figure 5.5: Tokenization

5.4.2 Stop Words Removal

Stopwords are the most commonly occurring words in a text which do not provide any valuable information. Stopwords like they, there, this, where, etc are some of the stopwords. NLTK library is a common library that is used to remove stopwords and include approximately 180 stopwords which it removes.

msg_tokenied	no_stopwords
[go, until, jurong, point, crazy, vailable, only, in, bugis, n, great, world, la, e, buffet, cine, there, got, amore, wat]	[go, jurong, point, crazy, available, bugis, n, great, world, la, e, buffet, cine, got, amore, wat]
[ok, lar, joking, wif, u, oni]	[ok, lar, joking, wif, u, oni]
[free, entry, in, 2, a, wkly, comp, to, win, fa, cup, final, tkts, 21st, nay, 2005, text, fa, to, 87121, to, receive, entry, questionstd, txt, ratetcs, apply, 08452810075over18s]	[free, entry, 2, wkly, comp, win, fa, cup, final, tkts, 21st, may, 2005, text, fa, 87121, receive, entry, questionstd, txt, ratetcs, apply, 08452810075over18s]
[u, dun, say, so, early, hor, u, c, already, then, say]	[u, dun, say, early, hor, u, c, already, say]
[nah, i, dont, think, he, goes, to, usf, he, lives, around, here, though]	[nah, dont, think, goes, usf, lives, around, though]

Figure 5.6: Stop Words Removal Process

5.4.3 Stemming

It is also known as the text standardization step where the words are stemmed or diminished to their root/base form. For example, words like 'programmer', 'programming, 'program' will be stemmed to 'program'.

no_stopwords	msg_stemmed
[go, jurong, point, crazy, available, bugis, n, great, world, la, e, buffet, cine, got, amore, wat]	[go, jurong, point, crazi, avail, bugi, n, great, world, la, e, buffet, cine, got, amor, wat]
[ok, lar, joking, wif, u, oni]	[ok, lar, joke, wif, u, oni]
[free, entry, 2, wkly, comp, win, fa, cup, final, tkts, 21st, may, 2005, text, fa, 87121, receive, entry, questionstd, txt, ratetcs, apply, 08452810075over18s]	[free, entri, 2, wkli, comp, win, fa, cup, final, tkt, 21st, may, 2005, text, fa, 87121, receiv, entri, questionstd, txt, ratetc, appli, 08452810075over18]
[u, dun, say, early, hor, u, c, already, say]	[u, dun, say, earli, hor, u, c, alreadi, say]
[nah, dont, think, goes, usf, lives, around, though]	[nah, dont, think, goe, usf, live, around, though]

Figure 5.7: Stemming Process

5.4.4 Removing punctuation

It is also a technique of text pre-processing where we try to remove unnecessary punctuation symbols because their presence does not make any significance in our text data. For Example, 'yippee' and 'yippee!' are conveying the feeling of happiness and excitement and this exclamation mark is of no use here. We will remove punctuation marks from string.punctuation But you always add more symbols based on your use case.

5.5 Vectorization

Vectorization is jargon for a classic approach of converting input data from its raw format (i.e. text) into vectors of real numbers which is the format that ML models support. This approach will be used in our system as it has worked wonderfully across various domains, and it's now used in Machine Learning. In ML, vectorization is a step in feature extraction. The idea is to get some distinct features out of the text for the model to train on, by converting text to numerical vectors.

5.6 Algorithmns And Models

As above mentioned, various algorithms and models will be developed for building this spam detection system. Following topics describes all the algorithms and models that will be tested:

5.6.1 Naïve Bayes Classifier(NB)

Naïve Bayes algorithm is a supervised learning algorithm, which is based on Bayes theorem and used for solving classification problems. It is mainly used in text classification that includes a high-dimensional training dataset. Naïve Bayes Classifier is one of the simple and most effective Classification algorithms which helps in building the fast machine learning models that can make quick predictions. It is a probabilistic classifier, which means it predicts on the basis of the probability of an object. Some popular examples of Naïve Bayes Algorithm are spam filtration, Sentimental analysis, and classifying articles.Working of Naïve Bayes' Classifier include:

- Convert the given dataset into frequency tables.
- Generate Likelihood table by finding the probabilities of given features.
- Now, use Bayes theorem to calculate the posterior probability.

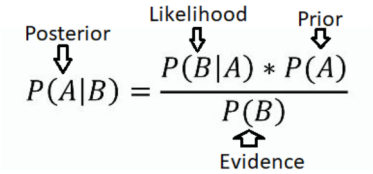

Figure 5.8: Bayes Formula

5.6.2 K-Nearest Neighbours(KNN)

For message classification (spam or ham), the features to be compared are the frequencies of words in each message. The euclidean distance is used to determine the similarity between two message; the smaller the distance, the more similar. The Euclidean Distance formula used in the algorithm is as follows :

Once the Euclidean Distance between a test message and each training message is calculated, the distances are sorted in ascending order (nearest to farthest), and the K-nearest neighbouring message are selected. If the majority is spam, then the test message is labelled as spam, else, it is labelled as ham. K-Nearest Neighbours(KNN Algorithm):

- Load the spam and ham message

$$D(a,b) = \sqrt{\sum_{i=1}^{n}(b_i - a_i)^2}$$

Where **a** is a test email with **n** features (words), and **b** is a training email with **n** features. The Euclidean distance is equal to the square root of the sum of the squared difference of features $\mathbf{b_i}$ and $\mathbf{a_i}$, where $\mathbf{b_i}$ is a word from email **b,** and $\mathbf{a_i}$ is a word from email **a**, and **i** is the word's number between 1 and **n.**

Figure 5.9: Euclidean Distance

- Remove common punctuation and symbols
- Lowercase all letters
- Remove stopwords (very common words like pronouns, articles, etc.)
- Split message into training message and testing message
- For each test message, calculate the similarity between it and all training message
- For each word that exists in either test message or training message, count its frequency in both message
- calculate the euclidean distance between both message to determine similarity
- Sort the message in ascending order of euclidean distance
- Select the k nearest neighbors (shortest distance)
- Assign the class which is most frequent in the selected k nearest neighbours to the new message

5.6.3 Random Forest(RF)

Random forest is a supervised learning algorithm which is used for both classification as well as regression. But however, it is mainly used for classification problems. As we know that a forest is made up of trees and more trees means more robust forest. Similarly, random forest algorithm creates decision trees on data samples and then gets the prediction from each of them and finally selects the best solution by means of voting. It is an ensemble method which is better than a single decision tree because it reduces the over-fitting by averaging the result.

- First, start with the selection of random samples from a given dataset.
- Next, this algorithm will construct a decision tree for every sample. Then it will get the prediction result from every decision tree.
- In this step, voting will be performed for every predicted result.
- At last, select the most voted prediction result as the final prediction result.

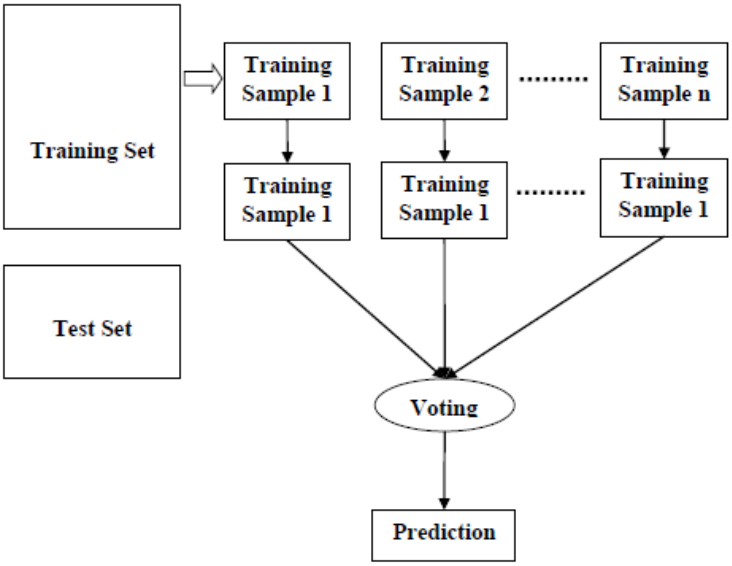

Figure 5.10: RF Algorithm

5.6.4 Logistic Regression(LR)

Logistic regression is a supervised learning classification algorithm used to predict the probability of a target variable. The nature of target or dependent variable is dichotomous, which means there would be only two possible classes. In simple words, the dependent variable is binary in nature having data coded as either 1 (stands for success/yes) or 0 (stands for failure/no). Mathematically, a logistic regression model predicts P(Y=1) as a function of X. It is one of the simplest ML algorithms that can be used for various classification problems.

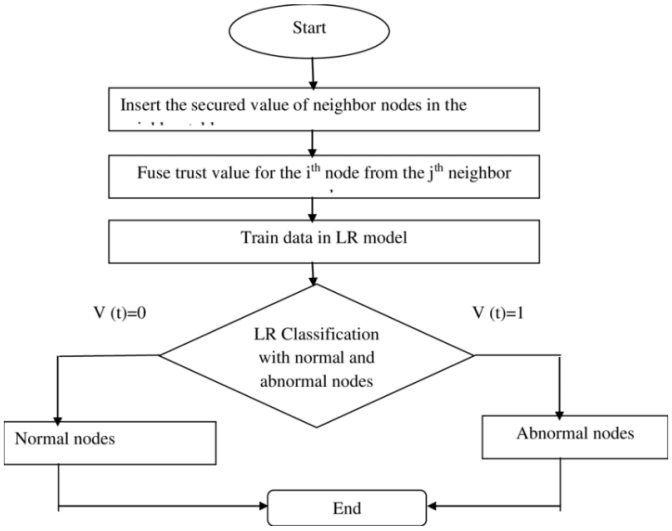

Figure 5.11: LR Flowchart

5.6.5 Support Vector Machine (SVM)

Support Vector Machine (SVM) is a supervised machine learning algorithm which can be used for both classification or regression challenges. However, it is mostly used in classification problems. In the SVM algorithm, we plot each data item as a point in n-dimensional space (where n is a number of features you have) with the value of each feature being the value of a particular coordinate. Then, we perform classification by finding the hyper-plane that differentiates the two classes very well.

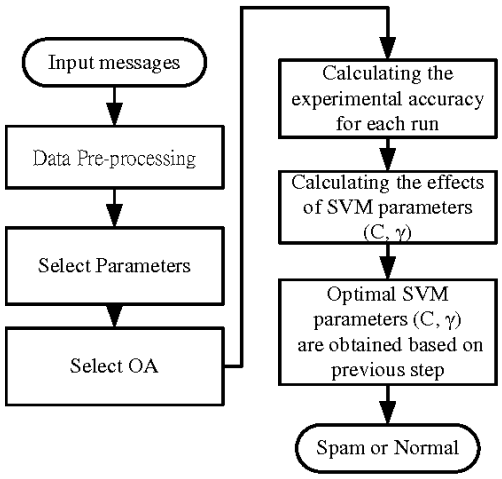

Figure 5.12: Support Vector Machine (SVM) Flowchart

5.6.6 Gradient Boosting Decision Trees (GBDT)

Gradient Boosting is a machine learning algorithm, used for both classification and regression problems. It works on the principle that many weak learners can together make a more accurate predictor.Gradient boosting works by building simpler (weak) prediction models sequentially where each model tries to predict the error left over by the previous model. Because of this, the algorithm tends to overfit rather quick. A model that does slightly better than random predictions.

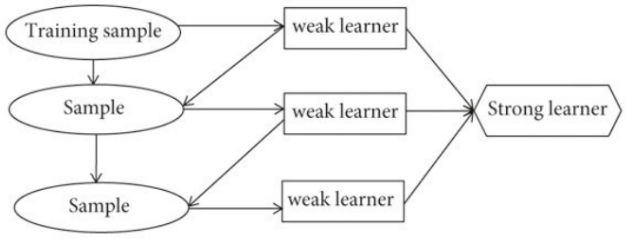

Figure 5.13: Gradient Boosting Decision Trees Flowchart

5.6.7 Extra Trees Classifier(ETC)

Extra trees (short for extremely randomized trees) is an ensemble supervised machine learning method that uses decision trees and is used by the Train Using AutoML tool. This method is similar to random forests but can be faster. The extra trees algorithm, like the random forests algorithm, creates many decision trees, but the sampling for each tree is random, without replacement. This creates a dataset for each tree with unique samples. A specific number of features, from the total set of features, are also selected randomly for each tree. The most important and unique characteristic of extra trees is the random selection of a splitting value for a feature. Instead of calculating a locally optimal value to split the data, the algorithm randomly selects a split value. This makes the trees diversified and uncorrelated.

5.7 Evaluation and Improvement

Since we will be testing different algorithms to train our model, we wiil be evaluating the accuracy and precision for each of those algorithms. As our dataset consists of inconsistent data (87.37% ham and 12.63% spam) priority will be given to precise model rather than accurate model.

After applying suitable algorithm to train our model, following steps will be performed to try to improve the accuracy and precision of our system:

- Change the max_features parameter of tf-idf
- Voting classifier
- Apply stacking

5.8 Deployment

The final product will be in the form of web-application which will then be deployed in Heroku.

Chapter 6

Cost Estimation

6.1 Team Member

Our team consists of 4 team members. Project was managed by using Clickup software.

6.2 Quality Control

Various testing mechanisms will be done in order to ensure the project meets the predefined and standardized quality standards. The cost estimation in order to carry out this will be dependent on the amount of testing that will be required to ensure that the project ultimately meets the requirements that are enlisted in the Requirement Analysis chapter.

6.3 Risk Management

During the course of designing and developing the project to counter the various risks that we might face, we will be developing a rudimentary working model of the final product before developing the actual product.

6.4 Material and Equipment

Following table consistes of Hardwares, Softwares and Tools requires for the project:

S.N.	Resources
1.	Computer
2.	Cloud Service
3.	Software
4.	GPU

Table 6.1: Materials and Equipment

6.5 Schedule

We have organized our schedule in above manner in order to effectively develop and complete our project.

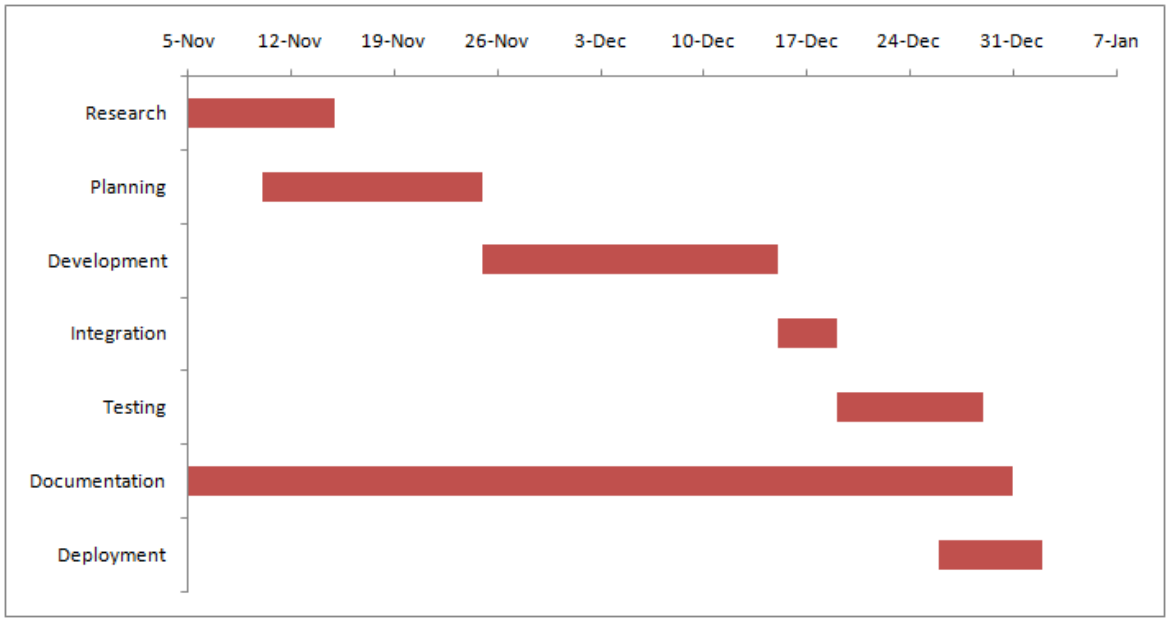

Figure 6.1: Gantt Chart

Chapter 7

Expected Outcomes

We are expecting the following accuracy and precision from the various algorithms that are to be tested to find out the best fitting algorithm for our system.

S.N.	Algorithm	Accuracy	Precison
1.	NB	95.93%	100.00%
2.	KNN	90.03%	100.00%
3.	RF	97.00%	99.08%
4.	LR	95.16%	94.00%
5.	SVM	97.29%	97.41%
6.	GBDT	95.16%	93.13%
7.	ETC	97.77%	99.14%

Table 7.1: Predicted Accuracy and Precision

After the completion of this project, following outcomes are expected:

- An online web application that is capable of taking any text//URL from user and finding out whether Ham or Spam.
- Error handling mechanisms will be added.
- A fully functional web application with beautiful and interactive UI will be developed
- The web application will be hosted in cloud service i.e. Heroku so that it accessed from any locations by multiple users at once via internet.

Bibliography

[1] M. A. Awad and M. Foqaha, "Email spam classification using hybrid approach of rbf neural network and particle swarm optimization," 2016.

[2] S. bin Abd Razak and A. F. B. Mohamad, "Identification of spam email based on information from email header," *2013 13th International Conference on Intellient Systems Design and Applications*, pp. 347–353, 2013.

[3] S. Ajaz, M. Nafis, and V. Sharma, "Spam mail detection using hybrid secure hash based naive classifier," 08 2022.

[4] S. B. Rathod and T. M. Pattewar, "Content based spam detection in email using bayesian classifier," in *2015 International Conference on Communications and Signal Processing (ICCSP)*, 2015, pp. 1257–1261.

[5] P. Sharma and U. Bhardwaj, "Machine learning based spam e-mail detection," *International Journal of Intelligent Engineering and Systems*, 2018.

[6] K. Agarwal and T. Kumar, "Email spam detection using integrated approach of naïve bayes and particle swarm optimization," in *2018 Second International Conference on Intelligent Computing and Control Systems (ICICCS)*, 2018, pp. 685–690.